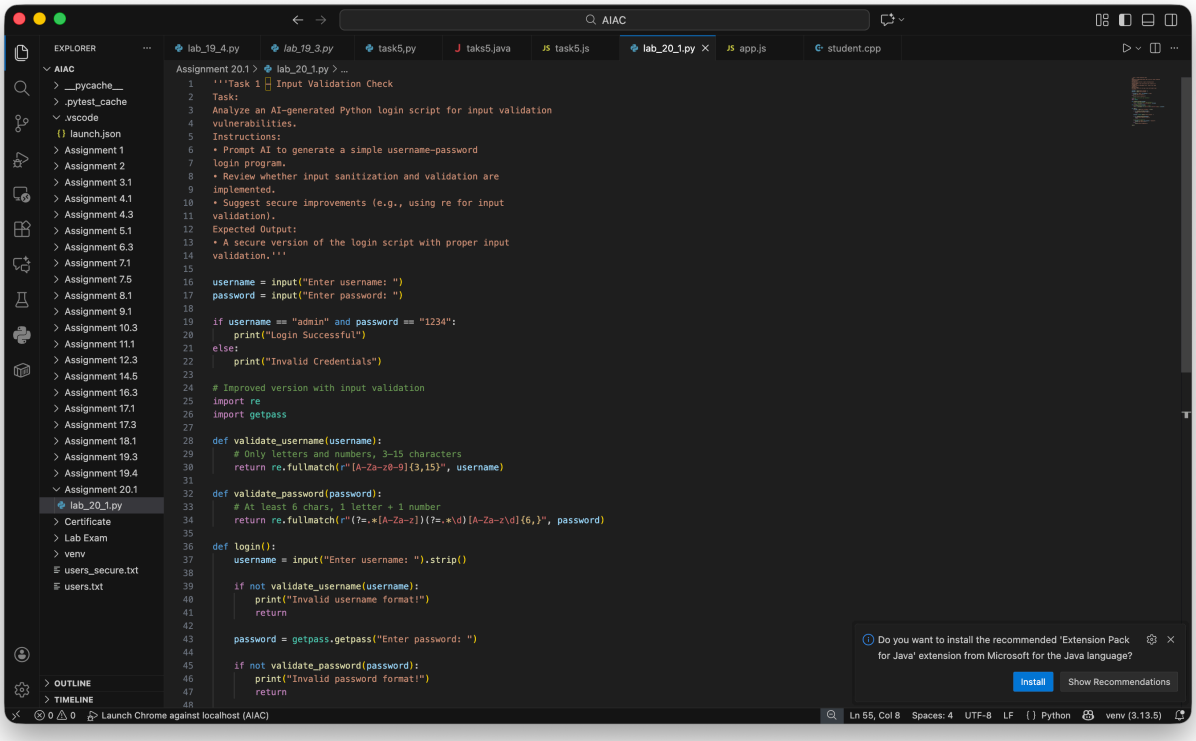
School of Computer Science and Artificial Intelligence

Lab Assignment - 20.1

Course Name : AI Assistant Coding
Name of the Student : Shaik Naved Ahmed
Registration No : 2303A54053
Batch No : 47-B

Task 1 – Input Validation Check
Task:
Analyze an AI-generated Python login script for input validation vulnerabilities.
Instructions:
• Prompt AI to generate a simple username-password login program.
• Review whether input sanitization and validation are implemented.
• Suggest secure improvements (e.g., using re for input validation).
Expected Output:
• A secure version of the login script with proper input validation.



Task 2 – SQL Injection Prevention
Task:
Test an AI-generated script that performs SQL queries on a database.
Instructions:
• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
• Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
• Refactor using parameterized queries (prepared statements).
Expected Output:
• A secure database query script resistant to SQL injection.

```python
#Task 2
'''Task 2 – SQL Injection Prevention
Task:
Test an AI-generated script that performs SQL queries on a
database.
Instructions:
• Ask AI to generate a Python script using SQLite/MySQL to
fetch user details.
• Identify if the code is vulnerable to SQL injection (e.g.,
using string concatenation in queries).
• Refactor using parameterized queries (prepared
statements).
Expected Output:
• A secure database query script resistant to SQL
injection.'''

import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

#  Vulnerable to SQL Injection
query = f"SELECT * FROM users WHERE username = '{username}'"
cursor.execute(query)

result = cursor.fetchall()

for row in result:
    print(row)

conn.close()
```

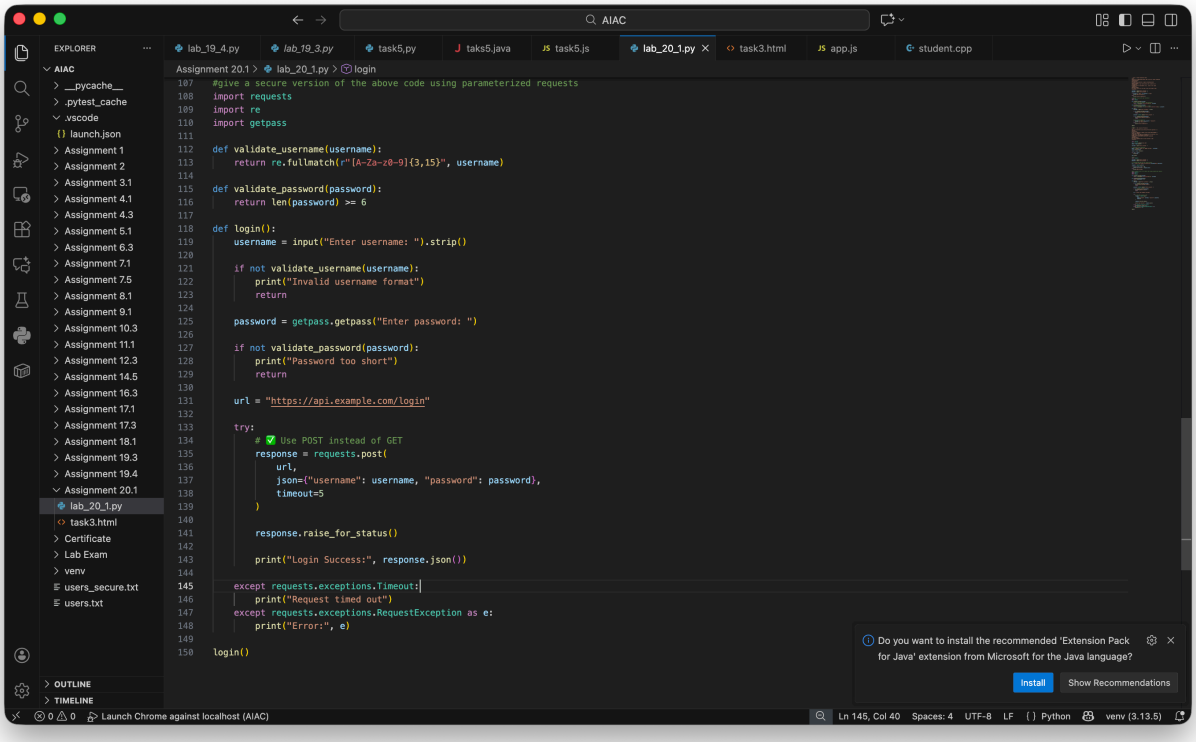
Task 3 – Cross-Site Scripting (XSS) Check
Task:
Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.
Instructions:
- **Ask AI to generate a feedback form with JavaScript-based output.**
- **Test whether untrusted inputs are directly rendered without escaping.**
- **Implement secure measures (e.g., escaping HTML entities, using CSP).**

Expected Output:
- **A secure form that prevents XSS attacks.**

```html
<html>
<body>

<script>
function showFeedback() {
    let userInput = document.getElementById("feedback").value;

    // ❌ Vulnerable to XSS
    document.getElementById("output").innerHTML = userInput;
}
</script>

</body>
</html>

<!--
This code is vulnerable to Cross-Site Scripting (XSS) attacks because it directly inserts user input into the HTML without any sanitization. An attacker could input malicio
-->
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';">
</head>
<body>

<h2>Feedback Form</h2>

<input type="text" id="feedback" placeholder="Enter feedback">
<button onclick="showFeedback()">Submit</button>

<p id="output"></p>

<script>
function escapeHTML(str) {
    return str
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
}

function showFeedback() {
    let userInput = document.getElementById("feedback").value;

    // ✅ Safe: Escaping input
    document.getElementById("output").innerHTML = escapeHTML(userInput);
```

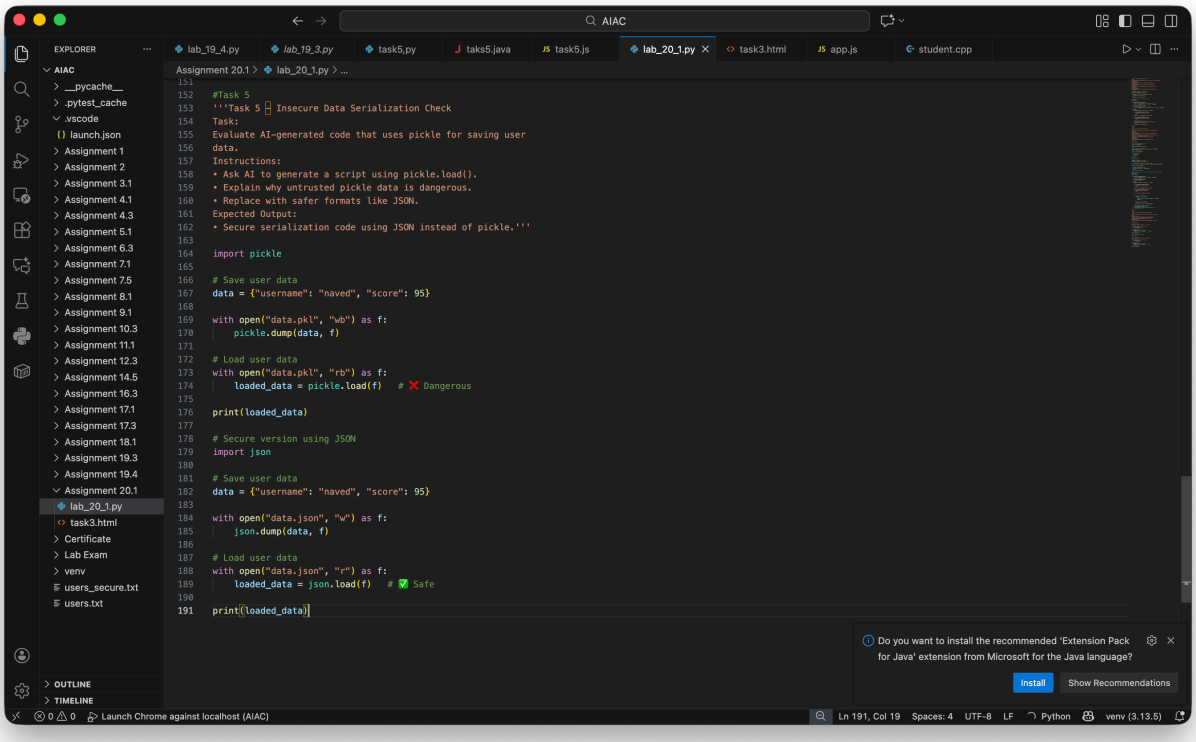
Feature	Insecure	Secure
Rendering	innerHTML	textContent
Input Handling	Raw	Escaped
XSS Risk	High	Prevented
CSP	Not used	Implemented

Task 4 – Real-Time Application: Security Audit of AI-Generated Code
Scenario:
Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).
Instructions:
• Perform a security audit to detect possible vulnerabilities.
• Prompt AI to suggest secure coding practices to fix issues.
• Compare insecure vs secure versions side by side.
Expected Output:
• A security-audited code snippet with documented vulnerabilities and fixes.



Feature	Insecure Code	Secure Code
Request Type	GET	POST
Password Handling	Visible	Hidden
Input Validation	None	Regex
Error Handling	None	try-except
Timeout	No	Yes
Data Exposure	High	Low

Task 5 – Insecure Data Serialization Check
Task:
Evaluate AI-generated code that uses pickle for saving user data.
Instructions:
• Ask AI to generate a script using pickle.load().
• Explain why untrusted pickle data is dangerous.
• Replace with safer formats like JSON.
Expected Output:
• Secure serialization code using JSON instead of pickle.



Feature	Pickle	JSON
Security	Unsafe	Safe
Code Execution Risk	High	None
Readability	Binary	Text
Portability	Python-only	Universal